CSIS 4495: Applied Research Project

Project Proposal

PowerShell-Based Active Directory Security Auditing Tool with a Local Plain-Language Remediation Dashboard

Prepared by: Hoang Nghiem Dac

Student ID: 300416011

Course: CSIS 4495: Applied Research Project, Section 002

Program: Post-Baccalaureate Diploma in Computer and Information Systems (Cybersecurity Option), Douglas College

Team Lead: Chris (individual project, no other team members)

Date: September 23, 2026

1. Introduction

1.1 Background and Context

Microsoft Active Directory (AD) remains the dominant identity and access management platform in enterprise IT, and it is consequently one of the most heavily targeted components of enterprise infrastructure. A joint cybersecurity advisory issued by the Australian Signals Directorate's Australian Cyber Security Centre (ASD ACSC), the U.S. Cybersecurity and Infrastructure Security Agency (CISA), the U.S. National Security Agency (NSA), the Canadian Centre for Cyber Security (CCCS), and the UK and New Zealand national cyber security centres describes AD's attack surface as "exceptionally large and difficult to defend," and documents 17 techniques commonly used by threat actors to compromise AD environments, including Kerberoasting, unconstrained delegation abuse, DCSync, and golden ticket attacks.

Small and medium-sized organizations frequently run AD environments without a dedicated security team. Their system administrators are usually competent at day-to-day AD administration but do not necessarily have the offensive-security background needed to interpret the output of specialist auditing tools or to prioritize remediation work.

1.2 Related Tools and Literature

Several established tools already assess AD security posture. PingCastle (originally developed by Vincent Le Toux, now maintained by Netwrix) produces a risk-scored health check across categories such as stale objects, privilege management, trust relationships, and anomalies. BloodHound takes a complementary, attack-path approach: it maps the graph of accounts, groups, and permissions inside AD and Entra ID to reveal how an attacker could chain existing privileges to reach Domain Admin, rather than scoring individual misconfigurations in isolation.

These tools are effective, but their output is written for an audience that already has red-team or blue-team security expertise. Attack-path graphs and configuration risk matrices assume the reader already knows what a Kerberoastable service account or an unconstrained delegation flag implies, and what to do about it. This is the specific gap this project intends to address: not by replacing PingCastle- or BloodHound-style detection logic, but by pairing a comparable detection engine with a plain-language explanation and remediation layer aimed at generalist administrators.

1.3 Problem Framing

The research question this project addresses is: can an open, PowerShell-native AD auditing tool detect a representative set of common AD misconfigurations and present each finding in a way that a generalist IT administrator, without dedicated security training, can understand and act on, without needing to consult a separate security specialist first? A secondary question is whether such a tool can be packaged so that another administrator can install and run it against their own AD environment, unmodified, rather than remaining a one-off lab demonstration.

1.4 Initial Hypotheses and Assumptions

- A representative subset of well-documented AD misconfigurations (stale accounts, weak password policy, over-privileged group membership, Kerberoasting exposure, unconstrained delegation, risky GPOs, domain trusts, LAPS deployment) can be reliably detected using native PowerShell and the ActiveDirectory module, without requiring a compiled binary or third-party agent.
- Separating the detection engine (PowerShell → structured JSON) from presentation (a local React dashboard) will make the tool more portable and reusable than a monolithic script, and will let the auditing engine be used headless from the command line or in a scheduled task.
- Framing each finding as "what is wrong / why it matters / what to do" will make the output more actionable for a non-specialist administrator than a raw technical report.
- Because AD audit output can reveal sensitive infrastructure information (account names, group structure, delegation paths), the presentation layer must run entirely locally, with no cloud dependency.

1.5 Potential Benefits

For the target user (a small IT team without a dedicated security function), this project's expected contribution is a free, self-hostable first line of AD security assessment that lowers the expertise barrier to acting on findings. For the author, the project is designed as a deliberate skills bridge from the author's existing web development background toward SOC analyst and cybersecurity engineering work, deepening PowerShell, Windows-based defensive security, and AD internals, an area the author's largely Linux-based coursework to date has not covered in depth.

2. Proposed Research Project

2.1 Target Users and Problem

The target users are small IT teams and system administrators responsible for Windows Active Directory environments who may not have a dedicated security team. AD environments accumulate security weaknesses over time: stale accounts, excessive privileges, weak password policies, exposed service accounts, unsafe delegation configurations, and other configuration drift. Existing auditing products can be powerful, but their results often require security expertise to interpret and convert into remediation actions.

The tool aims to make this process more accessible by answering three questions for every detected issue: What is wrong? Why does it matter? What should the administrator do about it?

2.2 Proposed Solution and Objectives

The objective is to design, build, and validate a PowerShell-based Active Directory security auditing tool that can be packaged and reused across different AD environments, with a local dashboard that translates technical findings into plain-language, prioritized remediation guidance. The initial set of planned checks includes:

- Stale and inactive accounts
- Password policy weaknesses
- Over-privileged group membership
- Kerberoasting exposure
- Unconstrained delegation
- Risky Group Policy configurations
- Domain trust relationships
- LAPS deployment/status

Each finding will contain a severity rating, technical evidence, a plain-language explanation of the security risk, and a recommended remediation action. If the complete set of planned checks proves too broad for the 12-week timeline, checks will be prioritized by security impact (favouring items called out in the CISA/NSA/ASD joint AD guidance, such as Kerberoasting and unconstrained delegation) while preserving the core architecture and deployment model.

2.3 Methodology and Justification

The methodology is design-and-build applied research: an initial detection engine is validated against a deliberately misconfigured Active Directory lab (built on the author's existing Windows Server 2022 AD home lab), then iteratively expanded and cross-checked against an established reference tool. This mirrors the approach taken by PingCastle and BloodHound's own development (build detection logic against known-bad configurations, then verify against real or realistic environments), while adding a remediation-explanation layer that neither existing tool provides for a generalist audience.

2.4 Data Collection, Sample Size, and Analysis

The primary "data" for this project is Active Directory configuration state, not survey or user data. The tool will be developed and validated against:

- A primary AD lab domain (Windows Server 2022) that the author already maintains, seeded with a representative set of known-bad configurations (stale accounts, over-privileged groups, a Kerberoastable service account, an unconstrained-delegation host, a weak password policy, etc.).
- At least one secondary, differently-configured AD lab domain built later in the project, used purely to test portability, i.e., that the tool runs correctly against an environment it was not developed against.

Detection accuracy will be analyzed by comparing the tool's findings against (a) the known, deliberately-seeded misconfigurations in the lab (ground truth) and (b) the output of PingCastle run against the same lab domain, to identify any findings PingCastle catches that the tool misses, and vice versa.

2.5 Technologies

Category	Choice and Rationale

Operating System / Platform	Windows Server 2022 domain controllers as the audit target; development and dashboard execution on Windows 11 with PowerShell 5.1 and PowerShell 7 (cross-version compatibility tested).
Programming Language(s) / Scripting / Framework	PowerShell (core detection engine, built on the ActiveDirectory RSAT module) for the auditing logic; TypeScript with React for the dashboard.
Database	No dedicated database in the core scope. Scan results are written as timestamped JSON snapshot files on the local filesystem, which keeps the module self-contained and avoids a persistence layer that would add maintenance overhead without serving a required feature. If time permits after the core checks are complete, a local SQLite store may be added as an optional extension to enable trend tracking of an environment's security posture across repeated scans.
Front-end and Back-end	Front-end: React + TypeScript single-page dashboard, served locally (no public/cloud deployment, since audit output can be sensitive). Back-end: the PowerShell module itself acts as the audit/back-end engine, producing structured JSON; a minimal local Node.js/Express bridge (or a static file server for the MVP) serves that JSON and the finding/remediation mapping layer to the dashboard.

2.6 Product Architecture

The product separates detection from presentation. The PowerShell module performs the audit and outputs structured JSON, so the auditing engine remains usable independently from the command line and does not depend on the dashboard. A separate mapping layer converts technical findings into human-readable descriptions and remediation guidance. A lightweight React dashboard then provides a local interface for reviewing results. Because AD audit results may contain sensitive infrastructure information, the dashboard operates locally rather than requiring a cloud service, and no scan data leaves the machine on which the audit is run. The intended workflow is:

AD Environment → PowerShell Audit Module → Structured JSON → Finding/Remediation Mapping → Local Dashboard

The local execution model is planned to mature in stages. For the Week 4 minimum viable product, the module writes a JSON file and the dashboard is served from a simple local static server and opened manually in a browser. During the packaging phase (Weeks 9 to 10), a single launcher script will run the scan, start the local server, and open the browser automatically, so that an administrator does not need command-line familiarity to use the tool. Packaging the dashboard as a standalone desktop application is noted as an optional extension only, and is not part of the committed scope.

2.7 Reusability and Distribution

Rather than being built specifically for the author's own lab environment, the tool is designed so that another administrator could deploy it against a compatible Active Directory environment without modifying the source code. The goal is to package the PowerShell component as a reusable module,

provide installation and usage documentation, define prerequisites and supported configurations, and make the project distributable through GitHub and potentially the PowerShell Gallery. This makes the final deliverable a software product that can be installed, executed, tested, and reused, rather than a one-time demonstration environment.

2.8 Expected Results

The expected outcome is a working, installable PowerShell module that can scan a live AD domain and produce a structured findings report, paired with a local dashboard that renders those findings with severity, plain-language explanation, and remediation guidance. The project should demonstrate that at least the eight planned check categories can be detected reliably in a realistic lab environment and that the tool can be run, unmodified, against a second AD environment it was not developed against. Practically, this contributes a free, low-friction "first pass" AD security assessment option for small IT teams that cannot justify commercial tooling or a dedicated security hire.

2.9 Evaluation Scheme

Validity, accuracy, and utility will be evaluated along three dimensions:

- Detection accuracy: for each check, compare tool output against the known, seeded ground-truth misconfigurations in the primary lab, and record false positives / false negatives. Where PingCastle covers an equivalent check, cross-validate against its findings for the same lab domain.
- Portability: run the packaged module against a second, differently-configured AD lab domain that the tool was not developed against, without modifying source code, and confirm it installs and produces correct output.
- Usability of remediation guidance: have at least one person without dedicated AD security expertise (e.g., a classmate or the author's existing IT contacts) read a sample of the tool's findings and give informal feedback on whether the "what's wrong / why it matters / what to do" explanation is understandable and actionable without further research.

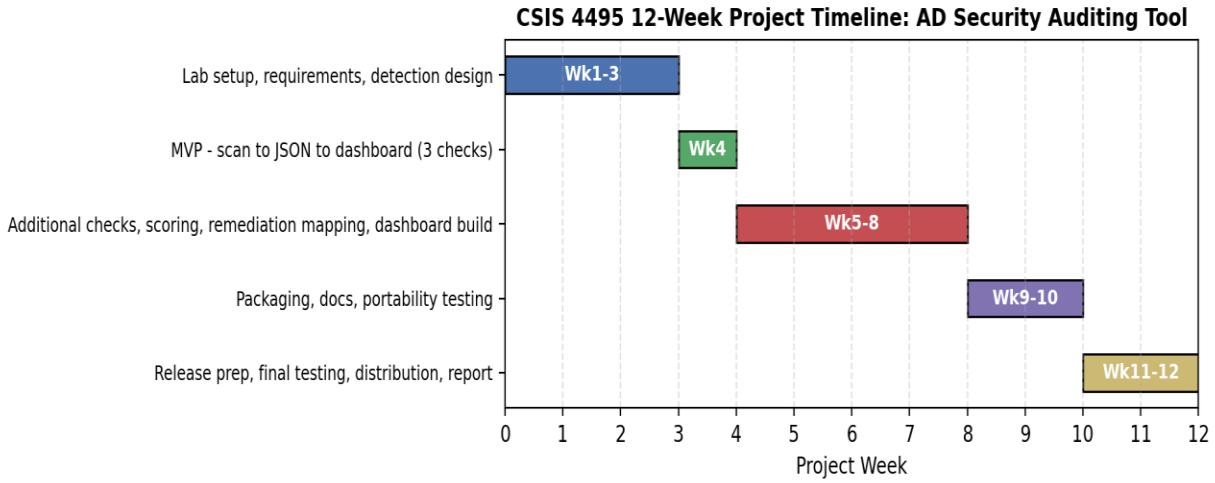
3. Project Planning and Timeline

This is an individually-completed applied research project (CSIS 4495 does not use teams); Chris is the sole author and is therefore responsible for all milestones and deliverables below.

3.1 Milestones

Weeks	Milestone / Deliverable
1–3	Week 1: finalize the check list and JSON output schema, scaffold the PowerShell module structure (.psm1 and .psd1 manifest). Week 2: build out the AD lab with seeded known-bad configurations, implement the first three checks (stale accounts, password policy, over-privileged group membership) and raw JSON export, and start the finding/remediation mapping entries for those checks. Week 3: build the React dashboard skeleton that reads the JSON and mapping layer and renders findings grouped by severity.

4	Minimum viable product: complete scan → JSON → dashboard workflow, with at least three security checks implemented end to end.
5–8	Weeks 5 to 6: implement the remaining checks (Kerberoasting exposure, unconstrained delegation, risky GPOs, domain trusts, LAPS status) and extend the finding/remediation mapping layer for each. Weeks 7 to 8: finalize the aggregator and severity/scoring model, validate detection against the seeded lab ground truth, and expand the dashboard with filtering, sorting, and expandable technical detail per finding.
9–10	Package the tool as a reusable PowerShell module (manifest, exported functions, versioning), build the local launcher script, write installation and usage documentation including a reference for every check, and run portability testing against a second, differently-configured AD lab.
11–12	Release preparation and final testing, GitHub / PowerShell Gallery distribution, final report writing, and presentation preparation.



3.2 Supervision and Check-ins

The author proposes a recurring weekly check-in with the course instructor/supervisor (video call or in-person, per the instructor's preference) to review progress against the milestones above, plus asynchronous updates via email or the course's designated communication channel between meetings.

4. Project Contract

I, Hoang Nghiem Dac, agree to complete the scope of work described in Section 2 (Proposed Research Project) according to the timeline and milestones set out in Section 3, for CSIS 4495. As this is an individually-completed project, I am solely responsible for all deliverables and progress reporting.

Additional agreements:

- Meeting frequency: weekly check-in with the course instructor/supervisor.
- Progress reporting: individual progress reports submitted as required by the course, plus the work log in Section 6 of this document, updated as work is completed.

- Scope changes: if the full set of planned checks proves too broad for the timeline, checks will be reprioritized by security impact while preserving the architecture and deliverable described in Section 2, in consultation with the instructor.

Signed: Hoang Nghiem Dac

Date: September 23, 2026

5. AI Use Section

5.1 Table of AI Tools and Specific Use

AI Tool Name	Version, Account Type	Specific Feature / Use	Value Addition
Claude (Anthropic)	*Claude Sonnet 5, account type TBD*	Used to help structure and draft this project proposal document (Introduction, timeline table, AI-use and work-log templates) from my own project description, milestones, and lab notes.	I reviewed, fact-checked, and edited the drafted sections against my own project plan; I supplied all project-specific decisions (check list, architecture, milestones) myself and used the tool for structuring, wording, and the timeline visualization rather than for the underlying technical design.

Note: please confirm/correct the account type above (free vs. paid) and add any additional AI tools used (e.g., for coding assistance during implementation) as the project progresses.

5.2 Appendix: Prompt History

Prompt history for the drafting session(s) referenced above will be exported and attached as an appendix, as required.

6. Work Date/Hours Log

Student Name: Chris

Per the course requirement, entries should be added every day work is done, in small, granular increments (not broad single entries of 10–12 hours). The sample entries below are illustrative only. They show the expected level of granularity and a typical load of roughly ten to twelve hours per week, and will be replaced with real dated entries as work is completed. Actual weekly hours will vary with the author's part-time work schedule.

Date	Hours	Description of Work Done
Sep 24, 2026	*2.0*	*Reviewed the CISA/NSA/ASD joint Active Directory guidance; drafted the initial check list*

Sep 25, 2026	*1.5*	*Reviewed PingCastle and BloodHound output formats as a reference for the findings schema*
Sep 26, 2026	*2.5*	*Drafted the JSON output schema for findings (severity, evidence, explanation, remediation)*
Sep 28, 2026	*2.0*	*Scaffolded the PowerShell module structure (.psm1 and .psd1 manifest); set up the Git repository*
Sep 29, 2026	*2.0*	*Snapshotted the AD lab VM; seeded stale accounts and a deliberately weak password policy*
Sep 30, 2026	*2.5*	*Implemented the stale and inactive account check; initial testing against the lab domain*
Oct 1, 2026	*3.0*	*Implemented the password policy check; added Pester tests for both checks*
Oct 2, 2026	*2.0*	*Drafted finding and remediation mapping entries for the first three checks*

7. Closing and References

7.1 Acknowledgements

The author thanks the CSIS 4495 course instructor for guidance on scoping this project toward a reusable, distributable product rather than a one-time lab demonstration.

7.2 References

Security Agency, National Security Agency, Canadian Centre for Cyber Security, National Cyber Security Centre (UK), & National Cyber Security Centre (NZ). (2024). *Detecting and mitigating Active Directory compromises*. Cybersecurity and Infrastructure Security Agency. https://www.cisa.gov/resources-tools/resources/detecting-and-mitigating-active-directory-compromises

Granneman, J. (n.d.). *How the BloodHound tool can improve Active Directory security*. TechTarget. Retrieved September 27, 2026, from https://www.techtarget.com/cybersecurity/tip/How-the-BloodHound-tool-can-improve-Active-Directory-security

Le Toux, V., & Netwrix. (n.d.). *PingCastle* [Computer software]. GitHub. Retrieved September 27, 2026, from https://github.com/netwrix/pingcastle

Microsoft. (n.d.). *Active Directory Domain Services documentation*. Microsoft Learn. Retrieved September 27, 2026, from https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/active-directory-domain-services

Netwrix. (n.d.). *PingCastle vs. BloodHound: AD attack path analysis*. Netwrix Blog. Retrieved September 27, 2026, from https://www.netwrix.com/en/resources/blog/pingcastle-vs-bloodhound/